Distributed Systems
COMP 212

Revision 1

Othon Michail

UNIVERSITY OF
LIVERPOOL

Fundamentals

- What is a Distributed System?

- What is a Distributed System?

- It is a collection of independent computers that appears to its users as a single coherent system

- Mention some of the main goals of Distributed Systems

- Mention some of the main goals of Distributed Systems

- Easily connect users/resources
- Transparency
- Openness
- Reliability
- Performance
- Scalability

- What is a transparent Distributed System?

- What is a transparent Distributed System?

- It is one that appears to its users as if it were only a single computer system

- What is a scalable Distributed System?

- What is a scalable Distributed System?

- It is one that is capable of growing w.r.t. its size, the maximum distance between its participants (geographical), and the number of administrative domains in it

Distributed Algorithms: Model

Network of Processors

- Processors/Processes/Nodes
  - Computing entities
  - All usually execute the same algorithm
  - A common exception is a pre-elected leader (e.g. root-leader in case of a tree)
- Networks/Graphs
  - Connects the processors
  - Directed/Undirected
  - Examples: ring, tree, complete network
  - Processors communicate by interchanging messages through the links of the network

Synchronous Rounds

A round:

1. all nodes read incoming messages
2. all nodes update their state
3. all nodes generate new messages and put them in transit
4. all messages are transmitted over the channels and the next round begins

1-3: Local computation by processors

4: Transmission of messages handled by the network (this step could even come first)

- Alternatives are equivalent: Use the one that is more convenient to you and the given problem

Distributed Tasks/Problems Examples

Broadcast Example

- Assumptions:
  - Network: Any undirected connected network
  - Processors: Have unique ids, Have no other information about the network (e.g., don't know its size n), There is a predetermined root (or leader u_0)
  - u_0 has some information it wishes to send to all processors
    - e.g., a message $\langle M \rangle$

- Problem:
  - Give a distributed algorithm that broadcasts $\langle M \rangle$ to all processors
  - All processors must eventually output $\langle M \rangle$
  - Possibly all processors should additionally have terminate in the end
  - Possibly the algorithm should additionally output a constructed spanning tree of G

Leader Election in a Ring Example

- Assumptions:
  - Network: Directed ring
  - Processors: Have unique ids, Know that the network is a ring, Have no other information about the network (e.g., don't know its size n)
- Problem:
  - Give a distributed algorithm that elects a unique leader
  - All processors have a *leader* variable set to 0 initially
  - Eventually only one processor should set *leader* := 1
  - Possible additional requirements: termination, all nodes know the elected leader

Leader Election in General Networks Example

- Assumptions:
  - Network: Any directed strongly connected network
  - Processors: Have unique ids, Know the diameter *D*, Have no other information about the network (e.g., don't know its size n)
- Problem:
  - Give a distributed algorithm that elects a unique leader
  - All processors have a *leader* variable set to 0 initially
  - Eventually only one processor should set *leader* := 1
  - Possible additional requirements: termination, all nodes know the elected leader

Algorithms

- Such problems are solved by distributed algorithms
- A distributed algorithm:
  - Is executed (in lock step) by all processors
  - Processors may have some input (ids, other info) and have some initial state
  - EVERY processor reads incoming messages-updates state-generates outgoing messages
  - The network delivers the messages
  - The next round begins
  - Processors typically produce some output (e.g. *leader* = 0 or *leader* = 1, *maxInput* = 52, …)

Algorithms

- Informal description
  - Gives the main idea
    - in verbal form avoiding much technical notation
  - Clear and concise to provide an accurate highlight/sketch of how the algorithm works
  - A well-written informal description can even convince in some cases before providing the formal presentation
- Pseudocode
  - See examples in the lecture notes
  - Typically, a piece of code to be re-executed by all processors in every round
    - loop that breaks only with termination of the corresponding processor

Correctness and Performance

When we devise an algorithm we typically should

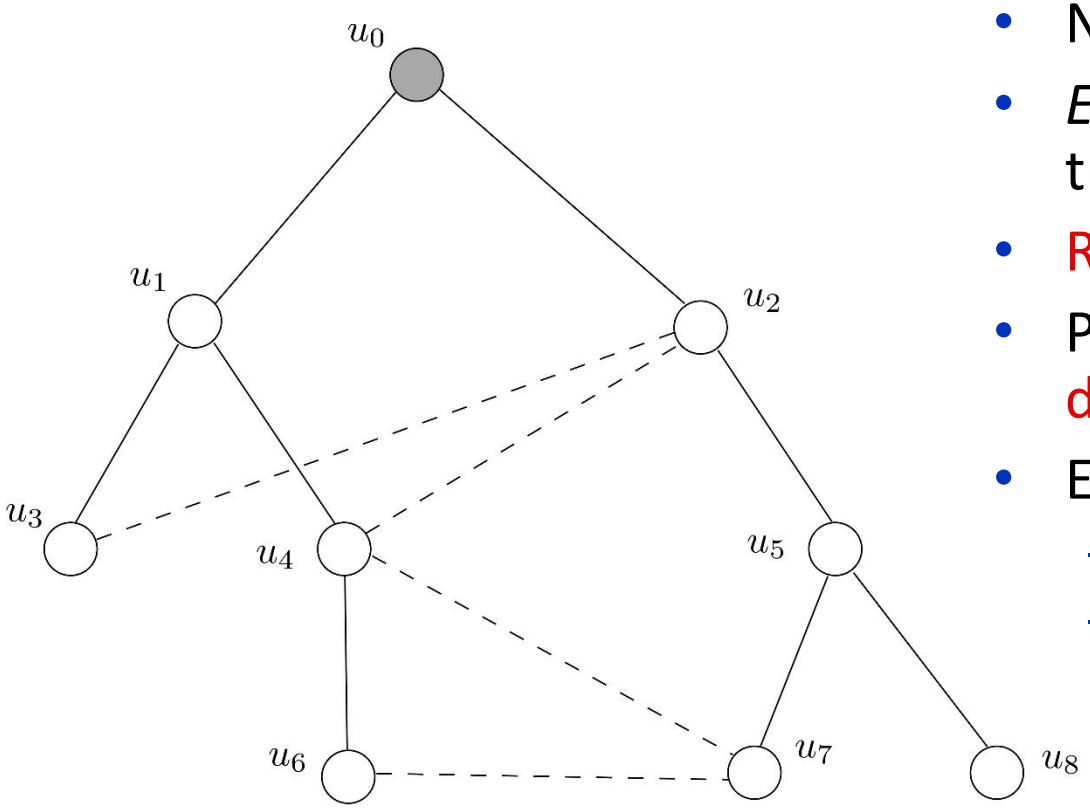
1. Convince that it is correct
2. Analyse its performance

- Correctness:
  - Usually a proof that the algorithm does as expected
- Performance:
  - Time Complexity (e.g., #rounds required)
  - Space Complexity (e.g., memory used by processors)
  - Communication Complexity (e.g., total #messages transmitted, size of messages)
  - If not possible/easy exactly, we do this asymptotically

Broadcast

Broadcast given Spanning Tree

- A spanning tree of the network is given



- Network $G = (V, E)$
- $E' \subseteq E$ specifies a spanning tree $T = (V, E')$
- Root: u_0 (leader)
- Processors know T in a distributed way
- Each u_i knows:
  - a $parent_i$
  - a set $children_i$

Broadcast given Spanning Tree

Problem:

- u_0 has some information it wishes to send to all processors
  - e.g., a message $\langle M \rangle$
  - additionally all nodes must have terminated in the end

Solution: Informal Description

- Root u_0 sends ⟨M⟩ to all channels leading to its children and terminates
- When a u_i receives ⟨M⟩ through the channel from its parent
  - it sends ⟨M⟩ to all channels leading to its children and
  - terminates

Spanning Tree Broadcast Correctness and Complexity

Correctness: By induction on r we prove that for every u_i whose distance from u_0 in the spanning tree is r, it holds that u_i receives $\langle M \rangle$ in round r.

Time Complexity: equal to the depth d of T

Communication Complexity:

- a total of n - 1 messages
- the maximum size of a message is equal to the binary representation of the information to be broadcast.

Broadcast without a given Spanning Tree

Problem:

- u_0 has some information it wishes to send to all processors
  - e.g., a message $\langle M \rangle$
  - additionally all nodes must have terminated in the end
- No spanning tree of the network G is given in advance
- The algorithm should also output a constructed spanning tree of G

Solution: Informal description

- All nodes awake initially
- If awake and have just received $\langle M \rangle$ from some neighbours,
  - Choose one of those neighbours as your *parent* and let him know
  - forward $\langle M \rangle$ to the rest of the neighbours
  - Wait for 1 round to collect children (if any) and then sleep
- If neighbours inform you that you are their parent,
  - add those processors to your *children* list
  - sleep
- If you are asleep, do nothing

Correctness and Complexity

- Correctness:
  - correctness of broadcast
  - correctness of spanning tree construction
    - can also be shown that the constructed tree is always a Breadth-first search (BFS) tree
- Time complexity:
  - $O(D)$: where D is the maximum distance of a u_i form u_0 in G
- Communication complexity:
  - size of messages: sends message M and an id
  - $O(m)$ messages: where m denotes the #edges of G

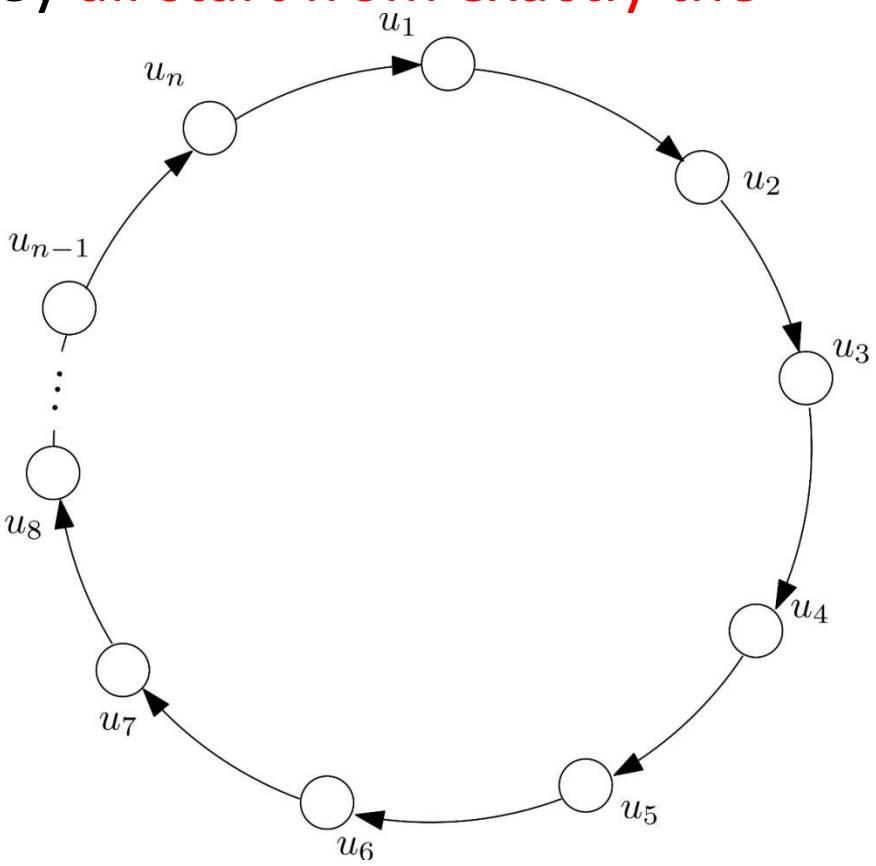
Leader Election

Problem Statement

- *Elect a unique leader processor from among all the processors in the distributed system*
- Leader to be interpreted as:
  - coordinator
  - master processor
- Special case of consensus/agreement
- Processors should agree eventually on who they elect

A First Minimal Setting

- Directed ring
- All processors are initially identical
  - Meaning here that they all start from exactly the same initial state

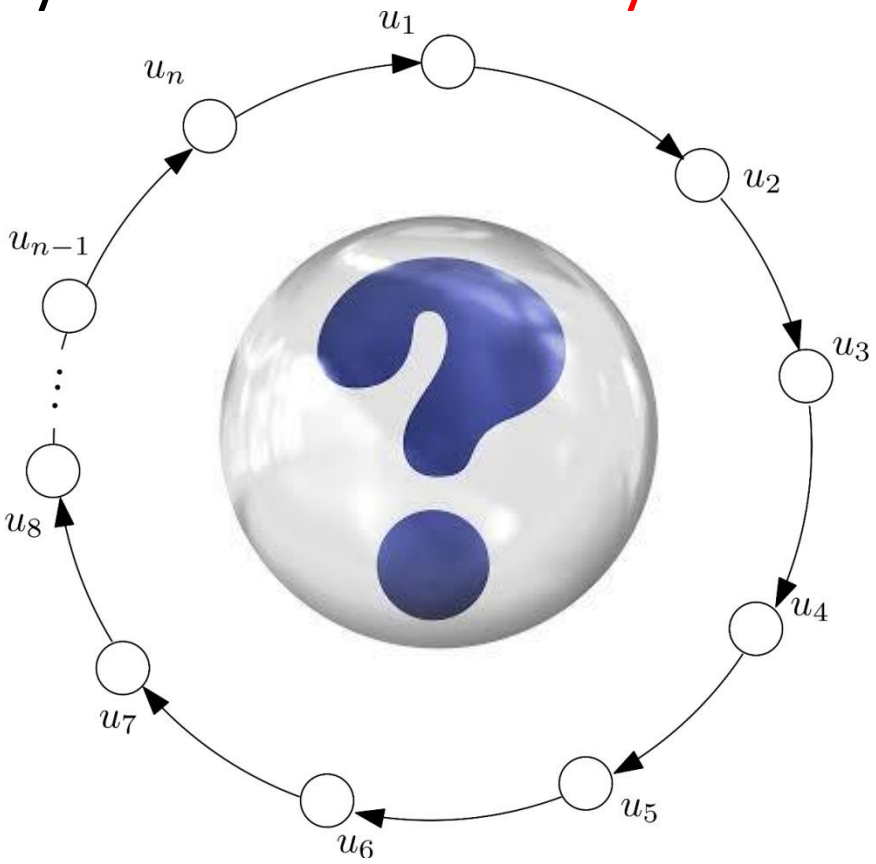


A First Minimal Setting

- Directed ring
- All processors are initially identical
  - Meaning here that they all start from exactly the same initial state

Question:

Is there an algorithm that solves leader election?



Our First Impossibility Result

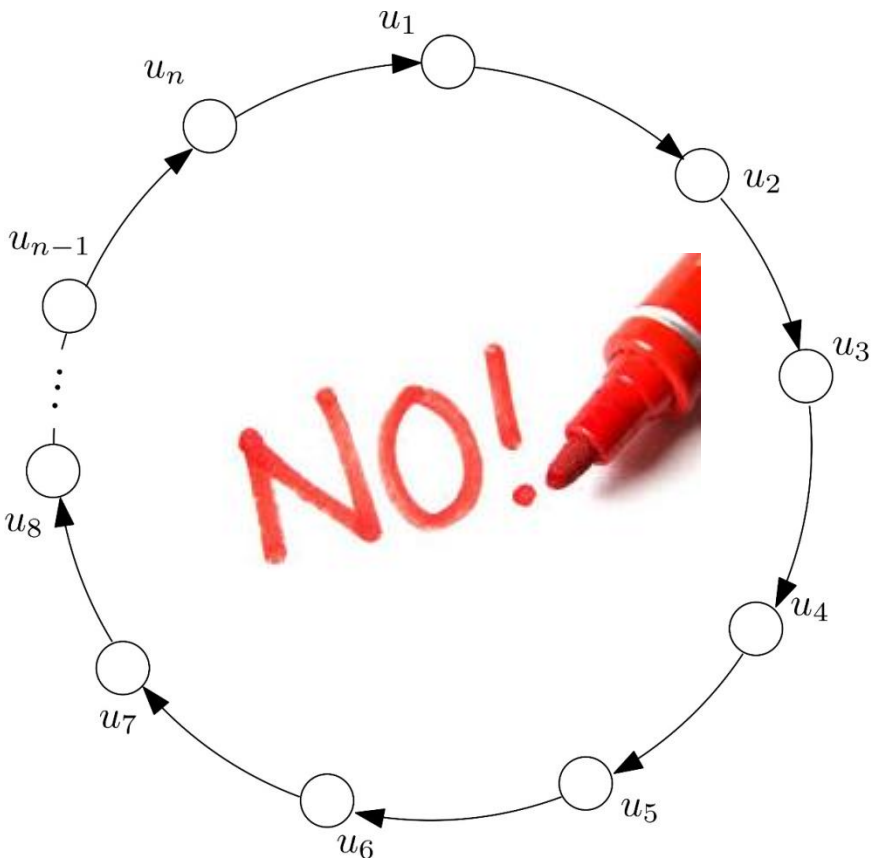
Question: *Is there an algorithm that solves leader election?*

Answer: No

Very general:

No matter which algorithm you try,

it will fail!



Leader Election with ids

- Processors have unique ids and do not know *n* in advance
- LCR algorithm: solves the problem
  - Le Lann, Chang, and Roberts [1977, 1979]
- Uses only transmission and comparison of ids
- Simplest version: Only the elected processor gives "output" and terminates
  - e.g., "I am the leader"
  - The other processors never produce any output and do not terminate

LCR: Informal description

- All processors send initially their id clockwise
- Upon receiving an incoming id, compare it to your own
  - If *incoming id* > *own id*, forward the received
  - if *incoming id* < *own id*, discard the received
  - if *incoming id* = *own id*, declare yourself the leader

- Intuitively:
  - The maximum id will manage to perform a complete turn and return to its origin
  - Any other id will at some point meet a processor with greater own id and will be discarded before making a complete turn

Correctness and Complexity

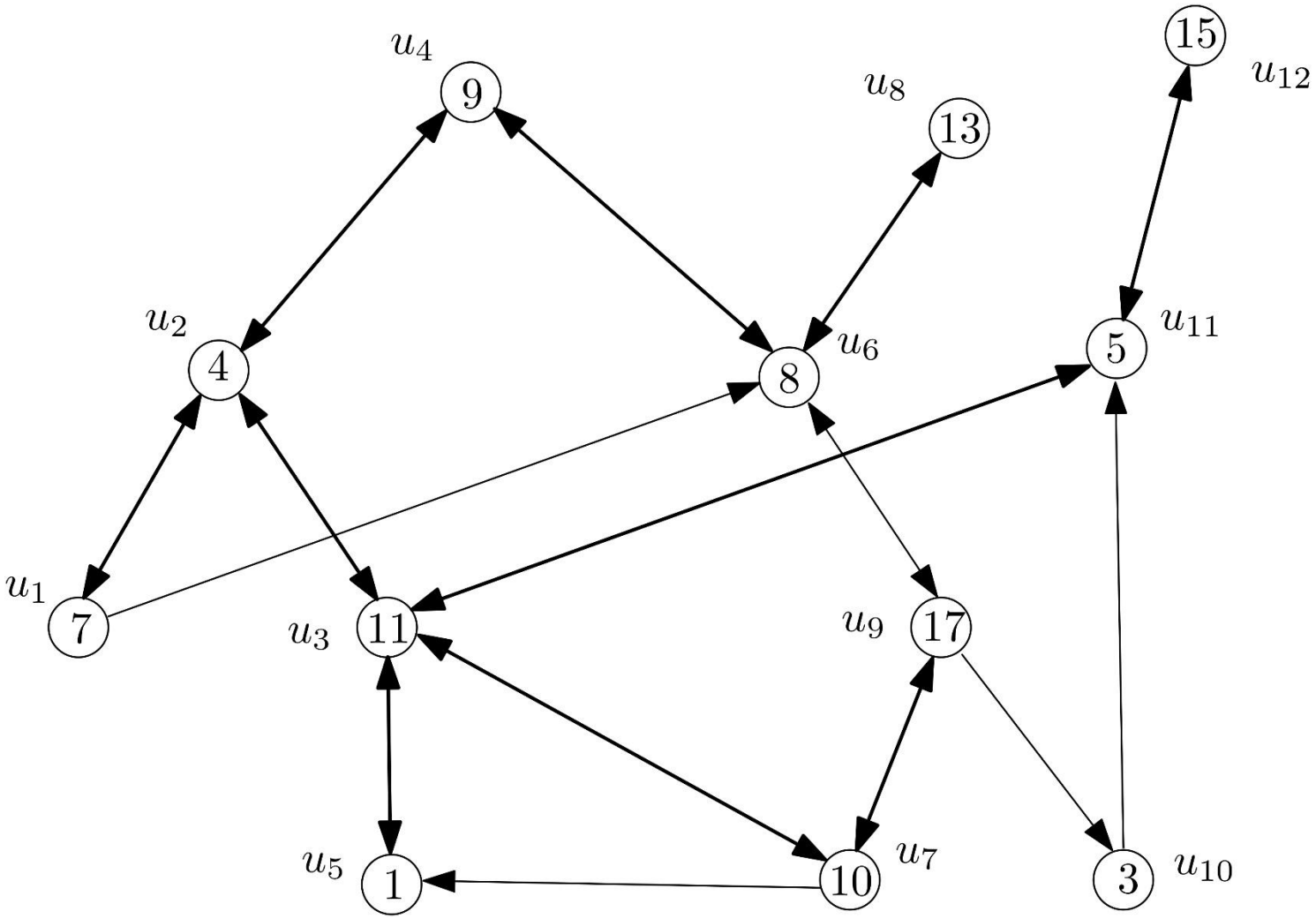
- Correctness:
  - some processor is eventually elected
  - never 2 or more processors are elected
- Time complexity:
  - n rounds
- Communication complexity:
  - size of messages: encoding in bits of the maximum id
  - $O(n^2)$ messages in the worst case
    - *Can you think which is the worst case for this algorithm?*
- *How can we make all nodes terminate and know the elected leader and what will be the additional effect in performance?*

Leader Election in General Networks

- *Elect a unique leader processor from among all the processors in the distributed system*
- Now the network can be any strongly connected directed network
  - Strongly connected: For every processors *u*, *v* there is a path from *u* to *v* and a path from *v* to *u*
  - e.g., the directed ring is just a special case
  - *Why can't we use LCR in this case?*
- Processors have unique ids

Leader Election in General Networks

- A strongly connected directed network



A Simple Algorithm based on Flooding

- Processors have unique ids, do not know *n* in advance, but do know the diameter *D* of the network
  - Diameter:
    - the distance between two nodes is given by the shortest path between them
    - Then the diameter of the network is determined by the pair of nodes at maximum distance (and is equal to that distance)
    - In other words, it is the maximum shortest path in the network
- FloodMax algorithm: solves the problem
- Uses transmission, comparison, and storage of ids
- Main idea: Flood the maximum id
  - LCR also does something like this but does not require knowledge of *D* and its termination condition works only for rings

FloodMax: Informal description

- All processors know the diameter *D* and their own id in advance
- All processors remember the greatest id that they have "heard" so far (initially their own)
- In every round all processors send the greatest known to all their out-neighbours
- After *D* rounds compare the largest heard to your own
  - if *greatest heard* = *own id*, declare yourself the leader
  - otherwise, declare yourself non-leader
- Intuitively:
  - The maximum id will manage to reach the whole network
  - So everyone non-maximum will know that there is a greater id and u_{max} can never receive a larger id

Correctness and Complexity

- Correctness:
  - exactly one processor is elected in the last round
- Time complexity:
  - $D + 1$ rounds (or D depending on the round model)
- Communication complexity:
  - size of messages: encoding in bits of the maximum id
  - $D \cdot m$ messages always
    - m is the number of directed links in the network

Communication

- Classify the protocols IP, TCP, HTTP, and FTP according to the ISO OSI layered classification

- Classify the protocols IP, TCP, HTTP, and FTP according to the ISO OSI layered classification

- IP: network protocol
- TCP: transport protocol
- HTTP, FTP: application protocols

- What are the main properties of TCP and UDP w.r.t. speed, reliability, connection/less, content transmitted?

- What are the main properties of TCP and UDP w.r.t. speed, reliability, connection/less, content transmitted?

- TCP (Transmission Control Protocol)
  - Connection-oriented
  - Reliable but slow (introduces overhead to guarantee delivery)
  - Applications: HTTP, POP, IMAP

- UDP (User Datagram Protocol)
  - Connection-less
  - Very fast but unreliable (does not guarantee delivery)
  - Applications: Domain Name System (DNS), Voice and Video streaming, VoIP

- What is a socket?

- What is a socket?
- A socket is one endpoint of a two-way communication link between two programs running on the network
- Processes bind sockets to a connection and read/write to them. A socket is bound to a port number (logical construct) so that the TCP layer can identify the application that data is destined to be sent.

- What is a Remote Procedure Call (RPC)? What do RPCs offer to the Distributed Systems programmer compared to socket-programming?

- What is a Remote Procedure Call (RPC)? What do RPCs offer to the Distributed Systems programmer compared to socket-programming?

- A Remote Procedure Call is a method that allows programs to call procedures located on other machines

- RPCs effectively remove the need for the Distributed Systems programmer to worry about all the details of network programming. As far as the programmer is concerned, a “remote” procedure call looks and works identically to a “local” procedure call. In this way, transparency is achieved.

- What are the steps involved in doing a remote computation through RPC?

- What are the steps involved in doing a a remote computation through RPC?

1. Client procedure calls client stub in a normal way
2. Client stub builds message, calls local OS
3. Client's OS sends message to remote OS
4. Remote OS gives message to server stub
5. Server stub unpacks parameters, calls server
6. Server does work, returns result to the stub
7. Server stub packs it in message, calls local OS
8. Server's OS sends message to client's OS
9. Client's OS gives message to client stub
10. Stub unpacks result, returns to client

- What is HTTP and what HTTPS?

- What is HTTP and what HTTPS?

- Hypertext Transfer Protocol
- Used to transmit resources on the WWW
  - HTML files, image files, query results, …
  - Identified by Uniform Resource Locators (URLs)
- Uses the client-server model
  - HTTP client: a browser
  - HTTP server: a Web server
- Application layer protocol presuming a reliable underlying transport layer protocol
  - Usually using TCP/IP sockets
- HTTPS: secure version of HTTP

- What is RMI?

- What is RMI?

- RMI: Remote Method Invocation
- Provides for remote communication between objects written in Java
- Allows an object running in one Java virtual machine to invoke methods on an object running in another Java virtual machine

- List in the correct order all the steps involved and the tools required in order to create and execute a client-server application using Remote Method Invocation (RMI)

- List in the correct order all the steps involved and the tools required in order to create and execute a client-server application using Remote Method Invocation (RMI)

1. Create the Interface to the server
2. Create the Server
3. Create the Client
4. Compile the Interface (javac)
5. Compile the Server (javac)
6. Compile the Client (javac)
7. Start the RMI registry (rmiregistry)
8. Start the RMI Server
9. Start the RMI Client

- Explain the difference between transient asynchronous communication and persistent synchronous communication

- Explain the difference between transient asynchronous communication and persistent synchronous communication

- In transient asynchronous communication, both the client and the server must be online at the same time. However, the client will continue to work while the server is processing the client request.

- On the other hand in persistent synchronous communication, the client and the server do not need to be online at the same time. Moreover, the client will wait until the server has processed the client's request and has responded.

- A client A would like to send a message to a server B. However, when A is sending the message, B might not be online to receive this message.
- Can we use message oriented middleware for sending this message from A to B? Explain.

- A client A would like to send a message to a server B. However, when A is sending the message, B might not be online to receive this message.
- Can we use message oriented middleware for sending this message from A to B? Explain.

- Yes. The message queues in MOM will store the message and deliver it to B when B becomes alive.

- Describe the purpose of message brokers in a message oriented middleware

- Describe the purpose of message brokers in a message oriented middleware

- Message brokers convert the data generated from different components (including legacy systems) into messages that can be communicated in MOM and stored in message queues
- They convert incoming messages in a format that can be understood by the destination application

- Mention two methods of reducing jitter in streaming applications

- Mention two methods of reducing jitter in streaming applications

- Buffering
- Distribute the frames of each packet into different packets

Processes

- What it means for a server to be stateless and what to be stateful?

- What it means for a server to be stateless and what to be stateful?

- Stateless servers: no information is maintained about the current “connections” to the server
  - e.g., the web
- Stateful servers: information is maintained about the current “connections” to the server
  - e.g., advanced file servers

- What is desktop virtualisation?

- What is desktop virtualisation?

- Desktop virtualisation, typically, allows one to run an entire (guest) operating system as a process within the (host) operating system controlling the hardware
  - e.g. Ubuntu Linux in VirtualBox running under Windows

Naming

- A client A from China wants to resolve the name http://www.csc.liv.ac.uk
  - Which service client A has to use?
  - Which is better for A w.r.t. efficiency: iterative or recursive name resolution?
  - How many name servers will be contacted directly by the client A and how many name servers will be involved?

- A client A from China wants to resolve the name http://www.csc.liv.ac.uk
  1. Which service client A has to use?
  2. Which is better for A w.r.t. efficiency: iterative or recursive name resolution?
  3. How many name servers will be contacted directly by the client A and how many name servers will be involved?

1. The Domain Name System (DNS)
2. Recursive: Only two long distance transmissions and the rest are short distance
3. 1 will be contacted directly by the client and 5 will be involved in total

- Mention the main functions of DNS

- Mention the main functions of DNS

- Resolve domain names for computers, i.e. to get their IP addresses
- Get mail host for a domain
- Reverse resolution - get domain name from IP address
- Host information - type of hardware and OS
- Well-known services - a list of well-known services offered by a host